

Manufacturing Yield Prediction Project

Comprehensive Technical Documentation & Machine Learning Workflow

1. Project Overview & Objective

- **Core Purpose:** Predict industrial manufacturing process quality outcomes as either **Pass** or **Fail** using real-time machine sensor metrics.
- **Problem Category:** Supervised Machine Learning → Binary Classification task.
- **Business Value:** Minimizes defective production output, catches line faults proactively, and reduces quality control operational overhead.

2. Dataset Architecture (`signal-data.csv`)

- **Dimensions:** Exactly 1,567 observations (rows) by 592 features (columns).
- **Dataset Layout Breakdown:**
 - **1 Timestamp Column (`Time`):** Records exact sample generation date/time.
 - **590 Sensor Features (`0` through `589`):** Numerical measurements capturing industrial equipment telemetry.
 - **1 Target Column (`Pass/Fail`):** Binary label indicating quality success (`-1`) or defect (`1`).

3. Target Distribution & Imbalance Analysis

- **Target Classes:**
 - `-1` (Pass): 1,463 observations (~93.4%)
 - `1` (Fail): 104 observations (~6.6%)
- **Imbalance Challenge:** A naive model predicting "Pass" every time achieves ~93% baseline accuracy without learning meaningful patterns.
- **Mitigation Strategy:** Applied **SMOTE** (Synthetic Minority Over-sampling Technique) to synthetically balance the training distribution.

4. Data Loading & Initial Inspection Code

```
import pandas as pd # Load dataset into DataFrame data = pd.read_csv('signal-data.csv')
# Inspect structure, metadata, summary statistics data.head() data.info()
data.describe(include='all')
```

5. Data Cleaning & Preprocessing Workflow

- **Missing Value Treatment:**
 - Sensors occasionally miss telemetry records, leading to `NaN` entries.
 - *Current Implementation:* Replaced missing values using `.fillna(value='0')`.
 - *Interview Consideration:* In production settings, median or model-based imputation is preferred over zero-filling.
- **Constant Feature Removal:**
 - Evaluated all columns for zero-variance parameters using column checks.
 - Removed all-zero columns where variance equaled zero, shrinking dimensionality from 592 down to **480 features**.
- **Duplicate Verification:** Validated row uniqueness via `.duplicated().sum()` , confirming zero completely duplicated rows.

6. Exploratory Data Analysis & Feature Insights

- **Correlation Analysis:** Linear relationships between individual features and `Pass/Fail` were weak (highest absolute correlations peaked around 0.156).
- **Takeaway:** Complex industrial failures rely on multi-feature interactions rather than isolated univariate signals.

7. Principal Component Analysis (PCA)

- **Objective:** Perform exploratory dimensionality reduction to visualize high-dimensional variance in 2D space.
- **Variance Captured:**
 - **PC1 (Principal Component 1):** Accounts for ~57.39% of data variance.
 - **PC2 (Principal Component 2):** Accounts for ~23.37% of data variance.
 - **Combined Variance:** First two components capture ~80.76% of total variance.
- **Technical Distinction:** PCA was utilized strictly for exploratory analysis and visualization; final classifiers were trained on standardized feature arrays.

Dimensionality Reduction

Reduces 480 cleaned sensor coordinates into orthogonal variance axes for scatter plotting and cluster observation.

Scaling Sensitivity

Raw sensor values span vastly different scales (thousands vs decimals), making feature standardization vital.

8. Feature Matrix & Target Splitting (`X` and `y`)

- **Feature Set (`X`):** Dropped the target column to retain independent sensor variables and numerical measurements.
- **Target Label (`y`):** Isolated the binary process outcome (`Pass/Fail`) for supervised target modeling.

9. Machine Learning Pipeline & Training Workflow

- **Class Imbalance Mitigation (SMOTE):**
 - Generated synthetic examples for the minority class ('1' - Fail) to create balanced training instances.
- **Train/Test Split:**
 - Split dataset into 80% training and 20% testing sets using `random\_state=42` for reproducibility.
- **Feature Scaling (StandardScaler):**
 - Normalized features to mean=0 and variance=1 via `scaler.fit\_transform()` on training data and `scaler.transform()` on test data.

10. Model Evaluation & Comparative Results

- **Evaluated Algorithms:** Support Vector Machine (SVM), Random Forest, and Gaussian Naive Bayes.
- **Comparative Accuracy:**
 - **SVM:** 99.15% test accuracy (Winner)
 - **Random Forest:** 98.98% test accuracy
 - **Naive Bayes:** 56.31% test accuracy
- **Why SVM Won:** Effective high-dimensional margin separation combined with feature scaling and balanced training classes.

11. Detailed Performance Metrics & Confusion Matrix

- **Classification Metrics:** Achieved strong precision, recall, and F1-scores (~0.99) across both classes in test evaluations.
- **Confusion Matrix Insight:** Validates accurate differentiation between genuine passing batches and critical manufacturing failures.

12. Model Persistence (`joblib`)

```
import joblib # Save trained classifier to disk joblib.dump(model,
'svm_yield_model.joblib') # Load model later for live batch inference loaded_model =
joblib.load('svm_yield_model.joblib')
```

13. Recommended Professional Improvements

- **Pipeline Leakage Fix:** Apply SMOTE strictly inside cross-validation loops or after train/test splitting to prevent data leakage.
- **Evaluation Beyond Accuracy:** Utilize ROC-AUC and Precision-Recall curves given the severe class imbalance.